

Chapter 6

C Arrays

Manipulate the basic data structure in C

Chapter Targets

- Use the array data structure to represent lists and tables of values
- Define an array, initialize an array, and refer to individual elements of an array
- Define symbolic constants
- Pass array to functions
- Use array to store, sort, and search lists and tables of values
- Define and manipulate multidimensional arrays
- Create variable-length arrays whose size is determined at execution time

Introduction

- This chapter serves as an introduction to **data structures**.
- Arrays are data structures consisting of related **data items of the same type**.
- In Chapter 10, we discuss C's notion of `struct` (structure)—a data structure consisting of related data items of possibly different types.
- Arrays and structures are “**static**” entities in that they **remain the same size** throughout program execution.

Arrays

- An array is a group of contiguous memory locations that all have the same type.
- To refer to a particular location or element in the array, we specify the array's name and the position number of the particular element in the array.
- A 12-element array for example:

All elements of this array share the array name, c →

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Position number of the element within array c

- Any one of these elements may be referred to by giving the array's name followed by the position number of the particular element in square brackets (`[]`).

Arrays (cont'd)

- The **first** element in every array is the **zeroth element**.
- An **array name**, like other **identifiers**, can contain only letters, digits and underscores and cannot begin with a digit.
- The **position number** within square brackets is called an **index or subscript**.
- An index must be an **integer** or an **integer expression**.

Arrays (cont'd)

- For example, the statement

```
c[2] = 1000;
```

assigns 1000 to array element `c[2]`.

- Similarly, if `a = 5` and `b = 6`, then the statement

```
c[a + b] += 2;
```

adds 2 to array element `c[11]`.

- An indexed array name is an **lvalue**—it can be used on the left side of an assignment.

Arrays (cont'd)

- Let's examine array `c` further

- The array's **name** is `c`.
- Its 12 elements are referred to as `c[0]`, `c[1]`, `c[2]`, ..., `c[10]` and `c[11]`.
- The **value** stored in `c[0]` is `-45`, the value of `c[1]` is `6`, `c[2]` is `0`, `c[7]` is `62` and `c[11]` is `78`.
- To print the sum of the values contained in the first three elements of array `c`, we'd write

```
printf("%d", c[0] + c[1] + c[2]);
```

- To divide the value of element 7 of array `c` by 2 and assign the result to the variable `x`, write

```
x = c[6] / 2;
```

All elements of this array share the array name, `c` →

<code>c[0]</code>	-45
<code>c[1]</code>	6
<code>c[2]</code>	0
<code>c[3]</code>	72
<code>c[4]</code>	1543
<code>c[5]</code>	-89
<code>c[6]</code>	0
<code>c[7]</code>	62
<code>c[8]</code>	-3
<code>c[9]</code>	1
<code>c[10]</code>	6453
<code>c[11]</code>	78

Position number of the element within array `c` →

Arrays.

- The **brackets []** used to enclose the index of an array are actually considered to be an **operator** in C.
- They have the same level of precedence as the **function call operator** (i.e., the parentheses that are placed after a function name to call that function).

Operators	Associativity	Type
[] () ++ (postfix) -- (postfix)	left to right	highest
+ - ! ++ (prefix) -- (prefix) (type)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Defining Arrays

- Arrays occupy space in memory.
- You specify the type of each element and the number of elements each array requires so that the computer may reserve the appropriate amount of memory.
- The following definition reserves 12 elements for integer array c, which has indices in the range 0 - 11.

```
int c[12];
```

Defining Arrays (cont'd)

- The definition

```
int b[100], x[27];
```

reserves 100 elements for integer array `b` and 27 elements for integer array `x`.

- These arrays have indices in the ranges 0–99 and 0–26, respectively.
- Arrays may contain other data types.
 - Character strings and their similarity to arrays are discussed in Chapter 8. The relationship between pointers and arrays is discussed in Chapter 7.

Array Examples

- Defining an Array and Using a Loop to Set the Array's Element Values
 - Like any other variables, uninitialized array elements contain **garbage values**.
 - Using for statements to initialize the elements of a five integer array `n` to zeros and print the array in tabular format

Array Examples (cont'd)

```
1 // Initializing the elements of an array to zeros.
2 #include <stdio.h>
3
4 // function main begins program execution
5 int main(void)
6 {
7     int n[5]; // n is an array of five integers
8
9     // set elements of array n to 0
10    for (size_t i = 0; i < 5; ++i) {
11        n[i] = 0; // set element at location i to 0
12    }
13
14    printf("%s%13s\n", "Element", "Value");
15
16    // output contents of array n in tabular format
17    for (size_t i = 0; i < 5; ++i) {
18        printf("%7u%13d\n", i, n[i]);
19    }
20
21    return 0;
22 }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Array Examples (cont'd)

- The first `printf` statement displays the column heads for the two columns printed in the subsequent for statement.
- Notice that the variable `i` is declared to be of type `size_t`, which represents an `unsigned integral` type according to the C standard.
 - ◆ This type is recommended for any variable that represents an `array's size` or an array's `indices`.
 - ◆ Type `size_t` is defined in header `<stddef.h>`, which is often included by other headers (such as `<stdio.h>`).
 - ◆ If you attempt to compile the program and receive errors, simply include `<stddef.h>` in your program.

Array Examples (cont'd)

- Initializing an Array in a Definition with an Initializer List
 - The elements of an array can also be **initialized when the array is defined** by following the definition with an **equals sign** and braces, `{}`, containing a comma-separated list of array initializers.
 - The following program initializes an integer array with five values and prints the array in tabular format.

Array Examples (cont'd)

```
1 // Initializing the elements of an array with an initializer list.
2 #include <stdio.h>
3
4 // function main begins program execution
5 int main(void)
6 {
7     // use initializer list to initialize array n
8     int n[5] = {32, 27, 64, 18, 95};
9
10    printf("%s%13s\n", "Element", "Value");
11
12    // output contents of array in tabular format
13    for (size_t i = 0; i < 5; ++i) {
14        printf("%7u%13d\n", i, n[i]);
15    }
16
17    return 0;
18 }
```

Element	Value
0	32
1	27
2	64
3	18
4	95

Array Examples (cont'd)

- If there are **fewer initializers** than elements in the array, the **remaining** elements are initialized to **zero**.
- For example, the elements of the array `n` could have been initialized to zero as follows:

```
// initializes entire array to zeros  
int n[10] = {0};
```

This **explicitly initializes the first element to zero** and initializes the remaining nine elements to zero because there are fewer initializers than there are elements in the array.

Array Examples (cont'd)

- Arrays are not automatically initialized to zero.
 - ◆ You must at least initialize the first element to zero for the remaining elements to be automatically zeroed.



Common Programming Error 6.1

Forgetting to initialize the elements of an array.

- Array elements are initialized
 - ◆ before program startup for static arrays, and
 - ◆ at runtime for automatic arrays.

Array Examples (cont'd)

- The array definition

```
int n[5] = {32, 27, 64, 18, 95, 14};
```

causes a **syntax error** because there are six initializers and only five array elements.



Common Programming Error 6.2

It's a syntax error to provide more initializers in an array initializer list than there are elements in the array—for example, `int n[3] = {32, 27, 64, 18};` is a syntax error, because there are four initializers but only three array elements.

- If the **array size** is omitted from a definition with an initializer list, the number of elements in the array will be the **number of elements in the initializer list**.

```
int n[] = {1, 2, 3, 4, 5};
```

would create a five-element array initialized with the indicated values.

Array Examples (cont'd)

- Specifying an Array's Size with a Symbolic Constant and Initializing Array Elements with Calculations
 - The following program initializes the elements of a 10-element array `s` to the values 2, 4, 6, ..., 20 and prints the array in tabular format.
 - The values are generated by multiplying the loop counter by 2 and adding 2.

Array Examples (cont'd)

```
1 // Initializing the elements of array s to the even integers from 2 to 20.
2 #include <stdio.h>
3 #define SIZE 5 // maximum size of array
4
5 // function main begins program execution
6 int main(void)
7 {
8     // symbolic constant SIZE can be used to specify array size
9     int s[SIZE]; // array s has SIZE elements
10
11     for (size_t j = 0; j < SIZE; ++j) { // set the values
12         s[j] = 2 + 2 * j;
13     }
14
15     printf("%s%13s\n", "Element", "Value");
16
17     // output contents of array s in tabular format
18     for (size_t j = 0; j < SIZE; ++j) {
19         printf("%7u%13d\n", j, s[j]);
20     }
21
22     return 0;
23 }
```

Element	Value
0	2
1	4
2	6
3	8
4	10

Array Examples (cont'd)

- The `#define` preprocessor directive is introduced in this program.

```
#define SIZE 5
```

defines a symbolic constant `SIZE` whose value is 5.

- A **symbolic constant** is an identifier that's replaced with **replacement text** by the C preprocessor before the program is compiled.
 - ◆ When the program is preprocessed, all occurrences of the symbolic constant `SIZE` are replaced with the replacement text `5`.

Array Examples (cont'd)

- Using symbolic constants to specify array sizes makes programs more **modifiable**.
 - We could have the first for loop fill a 1000-element array by simply changing the value of `SIZE` in the `#define` directive from 5 to 1000.
 - If the symbolic constant `SIZE` had not been used, we'd have to change the program in three separate places.



Software Engineering Observation 6.1

Defining the size of each array as a symbolic constant makes programs more modifiable.

Array Examples (cont'd)

- If the `#define` preprocessor directive is terminated with a semicolon, the preprocessor replaces all occurrences of the symbolic constant `SIZE` in the program with the text `5`.



Common Programming Error 6.3

Ending a `#define` or `#include` preprocessor directive with a semicolon. Remember that preprocessor directives are not C statements.



Common Programming Error 6.4

Assigning a value to a symbolic constant in an executable statement is a syntax error. The compiler does not reserve space for symbolic constants as it does for variables that hold values at execution time.



Good Programming Practice 6.1

Use only uppercase letters for symbolic constant names. This makes these constants stand out in a program and reminds you that symbolic constants are not variables.



Good Programming Practice 6.2

In multiword symbolic constant names, separate the words with underscores for readability.

Array Examples (cont'd)

- Summing the Elements of an Array
 - The next program sums the values contained in the 12-element integer array `a`.
 - The `for` statement's body does the totaling.

Array Examples (cont'd)

```
1 // Computing the sum of the elements of an array.
2 #include <stdio.h>
3 #define SIZE 12
4
5 // function main begins program execution
6 int main(void)
7 {
8     // use an initializer list to initialize the array
9     int a[SIZE] = { 1, 3, 5, 4, 7, 2, 99, 16, 45, 67, 89, 45 };
10    int total = 0; // sum of array
11
12    // sum contents of array a
13    for (size_t i = 0; i < SIZE; ++i) {
14        total += a[i];
15    }
16
17    printf("Total of array element values is %d\n", total);
18
19    return 0;
20 }
```

Total of array element values is 383

Array Examples (cont'd)

- Using Arrays to Summarize Survey Results

- Our next example uses arrays to summarize the results of data collected in a survey.

- Consider the problem statement.

Forty students were asked to rate the quality of the food in the student cafeteria on a scale of 1 to 10 (1 means awful and 10 means excellent). Place the 40 responses in an integer array and summarize the results of the poll.

- This is a typical array application
- We wish to summarize the number of responses of each type (i.e., 1 through 10).

Array Examples (cont'd)

```
1 // Analyzing a student poll.
2 #include <stdio.h>
3 #define RESPONSES_SIZE 40 // define array sizes
4 #define FREQUENCY_SIZE 11
5
6 // function main begins program execution
7 int main(void)
8 {
9     // initialize frequency counters to 0
10    int frequency[FREQUENCY_SIZE] = {0};
11
12    // place the survey responses in the responses array
13    int responses[RESPONSES_SIZE] = {1, 2, 6, 4, 8, 5, 9, 7, 8, 10,
14    1, 6, 3, 8, 6, 10, 3, 8, 2, 7, 6, 5, 7, 6, 8, 6, 7, 5, 6, 6,
15    5, 6, 7, 5, 6, 4, 8, 6, 8, 10};
16
17    // for each answer, select value of an element of array responses
18    // and use that value as an index in array frequency to
19    // determine element to increment
20    for (size_t answer = 0; answer < RESPONSES_SIZE; ++answer) {
21        ++frequency[responses[answer]];
22    }
23
24    // display results
25    printf("%s%17s\n", "Rating", "Frequency");
26
27    // output the frequencies in a tabular format
28    for (size_t rating = 1; rating < FREQUENCY_SIZE; ++rating) {
29        printf("%6d%17d\n", rating, frequency[rating]);
30    }
31
32    return 0;
33 }
```

Rating	Frequency
1	2
2	2
3	2
4	2
5	5
6	11
7	5
8	7
9	1
10	3

10	3
9	1
8	7
7	5
6	11
5	5
4	2
3	2
2	2
1	2

Array Examples (cont'd)

- The array responses is a 40-element array of the students' responses.
- We use an 11-element array frequency to count the number of occurrences of each response.
- We ignore frequency[0] because it's logical to have response 1 increment frequency[1] rather than frequency[0].
- This allows us to use each response directly as the index in the frequency array.



Good Programming Practice 6.3

Strive for program clarity. Sometimes it may be worthwhile to trade off the most efficient use of memory or processor time in favor of writing clearer programs.



Performance Tip 6.1

Sometimes performance considerations far outweigh clarity considerations.

```
int responses[RESPONSES_SIZE] = {1, 2, 6, 4, 8, 5, 9, 7, 8, 10,  
1, 6, 3, 8, 6, 10, 3, 8, 2, 7, 6, 5, 7, 6, 8, 6, 7, 5, 6, 6,  
5, 6, 7, 5, 6, 4, 8, 6, 8, 10};
```

Array Examples (cont'd)

- The for loop takes the responses one at a time from the array `responses` and increments one of the 10 counters (`frequency[1]` to `frequency[10]`) in the `frequency` array.

- The key statement in the loop is

```
++frequency[responses[answer]];
```

which increments the appropriate frequency counter depending on the value of `responses[answer]`.

- ◆ When the counter variable `answer` is 0, `responses[answer]` is 1, so `++frequency[responses[answer]]`; is interpreted as `++frequency[1]`; which increments array element one.
- ◆ When `answer` is 1, `responses[answer]` is 2, so `++frequency[responses[answer]]`; is interpreted as `++frequency[2]`; which increments array element two.

Array Examples (cont'd)

- C has no array bounds checking to prevent the program from referring to a non-existent element.
 - ◆ Regardless of the number of responses processed in the survey, only an 11-element array is required (ignoring element zero) to summarize the results.
 - ◆ If the data contained invalid values such as 13, the program would attempt to add 1 to frequency[13].
 - ◆ This would be outside the bounds of the array.
 - ◆ Actually, an executing program can “walk off” either end of an array without warning.



Common Programming Error 6.5

Referring to an element outside the array bounds.

- ◆ You should ensure that all array references remain within the bounds of the array.

Array Examples (cont'd)

◆ Error prevention when referring to array elements



Error-Prevention Tip 6.1

When looping through an array, the array index should never go below 0 and should always be less than the total number of elements in the array (size - 1). Make sure the loop-continuation condition prevents accessing elements outside this range.



Error-Prevention Tip 6.2

Programs should validate the correctness of all input values to prevent erroneous information from affecting a program's calculations.

Array Examples (cont'd)

- Graphing Array Element Values with Histograms
 - The next example reads numbers from an array and graphs the information in the form of a bar chart or histogram—each number is printed, then a bar consisting of that many asterisks is printed beside the number.
 - The nested for statement draws the bars.
 - Note the use of `puts("")` to end each histogram bar.

Array Examples

Element	Value	Histogram
0	19	*****
1	3	***
2	15	*****
3	7	*****
4	11	*****

```
1 // Displaying a histogram.
2 #include <stdio.h>
3 #define SIZE 5
4
5 // function main begins program execution
6 int main(void)
7 {
8     // use initializer list to initialize array n
9     int n[SIZE] = {19, 3, 15, 7, 11};
10
11     printf("%s%13s%17s\n", "Element", "Value", "Histogram");
12
13     // for each element of array n, output a bar of the histogram
14     for (size_t i = 0; i < SIZE; ++i) {
15         printf("%7u%13d", i, n[i]);
16
17         for (int j = 1; j <= n[i]; ++j) { // print one bar
18             printf("%c", '*');
19         }
20
21         puts(""); // end a histogram bar with a newline
22     }
23
24     return 0;
25 }
```

Array Examples (cont'd)

- Rolling a Die 60,000,000 Times and Summarizing the Results in an Array
 - Roll a single six-sided die 60,000,000 times to test whether the random number generator actually produces random numbers.
 - We will show an array version of this program.

Array Examples.

```
1 // Roll a six-sided die 60,000,000 times
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #define SIZE 7
6
7 // function main begins program execution
8 int main(void)
9 {
10     unsigned int frequency[SIZE] = {0}; // clear counts
11
12     srand(time(NULL)); // seed random number generator
13
14     // roll die 60,000,000 times
15     for (unsigned int roll = 1; roll <= 60000000; ++roll) {
16         size_t face = 1 + rand() % 6;
17         ++frequency[face]; // replaces entire switch
18     }
19
20     printf("%s%17s\n", "Face", "Frequency");
21
22     // output frequency elements 1-6 in tabular format
23     for (size_t face = 1; face < SIZE; ++face) {
24         printf("%4d%17d\n", face, frequency[face]);
25     }
26
27     return 0;
28 }
```

Face	Frequency
1	9997167
2	10003506
3	10001940
4	9995833
5	10000843
6	10000711



Using Character Arrays to Store and Manipulate Strings

- Array can be of any type.
- We now discuss storing strings in character arrays.
- So far, the only string-processing capability we have is outputting a string with `printf`.
- A string such as "hello" is really an array of individual characters in C.

Using Character Arrays to Store and Manipulate Strings

- Initializing a Character Array with a String

- A character array can be initialized using a string literal.
- For example,

```
char string1[] = "first";
```

initializes the elements of array `string1` to the individual characters in the string literal `"first"`.

- In this case, the size of array `string1` is determined by the compiler based on the length of the string.
- The string `"first"` contains five characters plus a special **string-termination character** called the **null character**.
 - ◆ Thus, array `string1` actually contains **six** elements.
 - ◆ The character constant representing the null character is `'\0'`.

Using Character Arrays to Store and Manipulate Strings

- All strings in C end with this character.
- A character array representing a string should always be defined large enough to hold the number of characters in the string and the terminating null character.
- Initializing a Character Array with an Initializer List of Characters
 - Character arrays also can be initialized with individual character constants in an initializer list, but this can be tedious.
 - The preceding definition is equivalent to

```
char string1[] = {'f', 'i', 'r', 's', 't', '\0'};
```

Using Character Arrays to Store and Manipulate Strings

- Accessing the Characters in a String
 - Because a string is really an array of characters, we can access individual characters in a string directly using array index notation.
 - For example, `string1[0]` is the character 'f' and `string1[3]` is the character 's'.

Using Character Arrays to Store and Manipulate Strings

- Inputting into a Character Array

- We can also input a string directly into a character array from the keyboard using `scanf` and the conversion specifier `%s`. For example,

```
char string2[20];
```

creates a character array capable of storing a string of at most 19 characters and a terminating null character.

- The statement

```
scanf("%19s", string2);
```

reads a string from the keyboard into `string2`.

- The name of the array is passed to `scanf` **without the preceding &** used with nonstring variables.
- The `&` is normally used to provide `scanf` with a **variable's location in memory** so that a value can be stored there.

Using Character Arrays to Store and Manipulate Strings

- When we discuss passing arrays to functions, we'll see that the value of an `array name` is the address of the start of the array; therefore, the `&` is not necessary.
- Function `scanf` will read characters until a `space`, `tab`, `newline` or `end-of-file` indicator is encountered.
- The `string2` should be no longer than 19 characters to leave room for the terminating null character.
- If the user types 20 or more characters, your program may crash or create a security vulnerability.
- For this reason, we used the conversion specifier `%19s` so that `scanf` reads a maximum of 19 characters and does not write characters into memory beyond the end of the array `string2`.

Using Character Arrays to Store and Manipulate Strings

- It's your responsibility to ensure that the array into which the string is read is capable of holding any string that the user types at the keyboard.
- Function `scanf` does not check how large the array is.
- Thus, `scanf` can write beyond the end of the array.

Using Character Arrays to Store and Manipulate Strings

- Outputting a Character Array that Represents a String
 - A character array representing a string can be output with `printf` and the `%s` conversion specifier.
 - The array `string2` is printed with the statement `printf("%s\n", string2);`
 - Function `printf`, like `scanf`, does not check how large the character array is.
 - The characters of the string are printed until a terminating null character is encountered.

Using Character Arrays to Store and Manipulate Strings

- Demonstrating Character Arrays
 - The following program demonstrates initializing a character array with a string literal, reading a string into a character array, printing a character array as a string and accessing individual characters of a string.

Using Character Arrays to Store and Manipulate Strings

```
1 // Treating character arrays as strings.
2 #include <stdio.h>
3 #define SIZE 20
4
5 // function main begins program execution
6 int main(void)
7 {
8     char string1[SIZE]; // reserves 20 characters
9     char string2[] = "string literal"; // reserves 15 characters
10
11     // read string from user into array string1
12     printf("%s", "Enter a string (no longer than 19 characters): ");
13     scanf("%19s", string1); // input no more than 19 characters
14
15     // output strings
16     printf("string1 is: %s\nstring2 is: %s\n"
17           "string1 with spaces between characters is:\n",
18           string1, string2);
19
20     // output characters until null character is reached
21     for (size_t i = 0; i < SIZE && string1[i] != '\0'; ++i) {
22         printf("%c ", string1[i]);
23     }
24
25     puts("");
26
27     return 0;
28 }
```

```
Enter a string (no longer than 19 characters): Hello there
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o
```

Static and Automatic Local Arrays

- Static Array

- Arrays that are static are initialized once at program startup.
- A static local variable exists for the duration of the program but is visible only in the function body.
- We can apply static to a local array definition so the array is not created and initialized each time the function is called and the array is not destroyed each time the function is exited in the program.
- This reduces program execution time, particularly for programs with frequently called functions that contain large arrays.
- If you do not explicitly initialize a static array, that array's elements are initialized to zero by default.



Performance Tip 6.2

In functions that contain automatic arrays where the function is in and out of scope frequently, make the array static so it's not created each time the function is called.

Static and Automatic Local Arrays (cont'd)

```
1 // Static arrays are initialized to zero if not explicitly initialized.
2 #include <stdio.h>
3
4 void staticArrayInit(void); // function prototype
5 void automaticArrayInit(void); // function prototype
6
7 // function main begins program execution
8 int main(void)
9 {
10     puts("First call to each function:");
11     staticArrayInit();
12     automaticArrayInit();
13
14     puts("\n\nSecond call to each function:");
15     staticArrayInit();
16     automaticArrayInit();
17
18     return 0;
19 }
```

Static and Automatic Local Arrays (cont'd)

```
21 // function to demonstrate a static local array
22 void staticArrayInit(void)
23 {
24     // initializes elements to 0 before the function is called
25     static int array1[3];
26
27     puts("\nValues on entering staticArrayInit:");
28
29     // output contents of array1
30     for (size_t i = 0; i <= 2; ++i) {
31         printf("array1[%u] = %d ", i, array1[i]);
32     }
33
34     puts("\nValues on exiting staticArrayInit:");
35
36     // modify and output contents of array1
37     for (size_t i = 0; i <= 2; ++i) {
38         printf("array1[%u] = %d ", i, array1[i] += 5);
39     }
40 }
```

Static and Automatic Local Arrays (cont'd)

```
42 // function to demonstrate an automatic local array
43 void automaticArrayInit(void)
44 {
45     // initializes elements each time function is called
46     int array2[3] = { 1, 2, 3 };
47
48     puts("\n\nValues on entering automaticArrayInit:");
49
50     // output contents of array2
51     for (size_t i = 0; i <= 2; ++i) {
52         printf("array2[%u] = %d ", i, array2[i]);
53     }
54
55     puts("\nValues on exiting automaticArrayInit:");
56
57     // modify and output contents of array2
58     for (size_t i = 0; i <= 2; ++i) {
59         printf("array2[%u] = %d ", i, array2[i] += 5);
60     }
61 }
```

Static and Automatic Local Arrays (cont'd)

First call to each function:

Values on entering staticArrayInit:

array1[0] = 0 array1[1] = 0 array1[2] = 0

Values on exiting staticArrayInit:

array1[0] = 5 array1[1] = 5 array1[2] = 5

Values on entering automaticArrayInit:

array2[0] = 1 array2[1] = 2 array2[2] = 3

Values on exiting automaticArrayInit:

array2[0] = 6 array2[1] = 7 array2[2] = 8

Second call to each function:

Values on entering staticArrayInit:

array1[0] = 5 array1[1] = 5 array1[2] = 5 — values preserved from last call

Values on exiting staticArrayInit:

array1[0] = 10 array1[1] = 10 array1[2] = 10

Values on entering automaticArrayInit:

array2[0] = 1 array2[1] = 2 array2[2] = 3 — values reinitialized after last call

Values on exiting automaticArrayInit:

array2[0] = 6 array2[1] = 7 array2[2] = 8

Static and Automatic Local Arrays (cont'd)

- Function `staticArrayInit` is called twice.
 - The local static array in the function is initialized to zero before program startup.
 - The function prints the array, adds 5 to each element and prints the array again.
 - The second time the function is called, the static array contains the values stored during the first function call.

Static and Automatic Local Arrays (cont'd)

- Function `automaticArrayInit` is also called twice.
 - The elements of the automatic local array in the function are initialized with the values 1, 2 and 3.
 - The function prints the array, adds 5 to each element and prints the array again.
 - The second time the function is called, the array elements are initialized to 1, 2 and 3 again because the array has automatic storage duration.



Common Programming Error 6.6

Assuming that elements of a local static array are initialized to zero every time the function in which the array is defined is called.

Passing Arrays to Functions

- To pass an array argument to a function, specify the array's name without any brackets.
- For example, if array `hourlyTemperatures` has been defined as

```
int hourlyTemperatures[HOURS_IN_A_DAY];
```

the function call

```
modifyArray(hourlyTemperatures, HOURS_IN_A_DAY)
```

passes array `hourlyTemperatures` and its size to function `modifyArray`.

Passing Arrays to Functions (cont'd)

- All arguments in C are passed by value.
- C automatically passes each array to functions by address to simulate passing argument by reference — the called functions can modify the element values in the callers' original arrays.
 - The name of the array evaluates to the **address** of the first element of the array.
 - Because the starting address of the array is passed, the called function knows precisely where the array is stored.



Software Engineering Observation 6.2

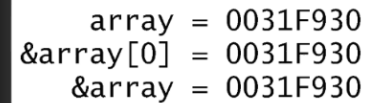
It's possible to pass an array by value (by placing it in a struct as we explain in Chapter 10, C Structures, Unions, Bit Manipulation and Enumerations).

Passing Arrays to Functions (cont'd)

- Therefore, when the called function modifies array elements in its function body, it's modifying the actual elements of the array in their original memory locations.
- We demonstrate that an array name is really the address of the first element of the array by printing `array`, `&array[0]` and `&array` using the `%p` conversion specifier—a special conversion specifier for printing addresses.
 - The `%p` conversion specifier normally **outputs addresses as hexadecimal numbers**, but this is compiler dependent.
 - Hexadecimal (base 16) numbers consist of the digits 0 through 9 and the letters A through F (these letters are the hexadecimal equivalents of the numbers 10–15).

Passing Arrays to Functions (cont'd)

```
1 // Array name is the same as the address of the array's first element.
2 #include <stdio.h>
3
4 // function main begins program execution
5 int main(void)
6 {
7     char array[5]; // define an array of size 5
8
9     printf("    array = %p\n&array[0] = %p\n    &array = %p\n",
10          array, &array[0], &array);
11
12     return 0;
13 }
```



```
array = 0031F930
&array[0] = 0031F930
&array = 0031F930
```

- The output shows that `array`, `&array` and `&array[0]` have the same value, namely 0012FF78.

Passing Arrays to Functions (cont'd)

- Although entire arrays are passed by reference, individual array elements are passed by value exactly as simple variables are.
- Such simple single pieces of data (such as individual ints, floats and chars) are called scalars.
- To pass an element of an array to a function, use the indexed name of the array element as an argument in the function call.
- In Chapter 7, we show how to pass **scalars** (i.e., individual variables and array elements) to functions by reference.



Performance Tip 6.3

Passing arrays by reference makes sense for performance reasons. If arrays were passed by value, a copy of each element would be passed. For large, frequently passed arrays, this would be time consuming and would consume storage for the copies of the arrays.

Passing Arrays to Functions (cont'd)

- For a function to receive an array through a function call, the function's parameter list must specify that an array will be received.
 - For example, the function header for function `modifyArray` (that we called earlier in this section) might be written as

```
void modifyArray(int b[], int size)
```

indicating that `modifyArray` expects to receive an array of integers in parameter `b` and the number of array elements in parameter `size`.
 - The **size** of the array is **not required** between the array brackets.

Passing Arrays to Functions (cont'd)

- If it's included, the compiler checks that it's greater than zero, then ignores it.
- Specifying a negative size is a compilation error.
- Because arrays are automatically passed by reference, when the called function uses the array name `b`, it will be referring to the array in the caller (array `hourlyTemperatures` in the preceding call).

Passing Arrays to Functions (cont'd)

- Difference Between Passing an Entire Array and Passing an Array Element
 - Next program demonstrates the difference between passing an entire array and passing an array element.
 - The program first prints the five elements of integer array `a`.
 - Next, `a` and its size are passed to function `modifyArray`, where each of `a`'s elements is multiplied by 2.
 - Then `a` is reprinted in `main`.

Passing Arrays to Functions

```
1 // Passing arrays and individual array elements to functions.
2 #include <stdio.h>
3 #define SIZE 5
4
5 // function prototypes
6 void modifyArray(int b[], size_t size);
7 void modifyElement(int e);
8
9 // function main begins program execution
10 int main(void)
11 {
12     int a[SIZE] = { 0, 1, 2, 3, 4 }; // initialize array a
13
14     puts("Effects of passing entire array by reference:\n\nThe "
15         "values of the original array are:");
16
17     // output original array
18     for (size_t i = 0; i < SIZE; ++i) {
19         printf("%3d", a[i]);
20     }
21
22     puts(""); // outputs a newline
23
24     modifyArray(a, SIZE); // pass array a to modifyArray by reference
25     puts("The values of the modified array are:");
26
27     // output modified array
28     for (size_t i = 0; i < SIZE; ++i) {
29         printf("%3d", a[i]);
30     }
31
32     // output value of a[3]
33     printf("\n\nEffects of passing array element "
34         "by value:\n\nThe value of a[3] is %d\n", a[3]);
35
36     modifyElement(a[3]); // pass array element a[3] by value
37
38     // output value of a[3]
39     printf("The value of a[3] is %d\n", a[3]);
40
41     return 0;
42 }
```

Effects of passing entire array by reference:

The values of the original array are:

0 1 2 3 4

The values of the modified array are:

0 2 4 6 8

Effects of passing array element by value:

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6

```
44 // in function modifyArray, "b" points to the original array "a"
45 // in memory
46 void modifyArray(int b[], size_t size)
47 {
48     // multiply each array element by 2
49     for (size_t j = 0; j < size; ++j) {
50         b[j] *= 2; // actually modifies original array
51     }
52 }
53
54 // in function modifyElement, "e" is a local copy of array element
55 // a[3] passed from main
56 void modifyElement(int e)
57 {
58     // multiply parameter by 2
59     printf("Value in modifyElement is %d\n", e *= 2);
60 }
```

Passing Arrays to Functions (cont'd)

- As the output shows, the elements of `a` are indeed modified by `modifyArray`.
- Now the program prints the value of `a[3]` and passes it to function `modifyElement`.
- Function `modifyElement` multiplies its argument by 2 and prints the new value.
- When `a[3]` is reprinted in main, it has not been modified, because individual array elements are passed by value.

Passing Arrays to Functions (cont'd)

- Using the `const` Qualifier with Array Parameters
 - There may be situations in your programs in which a function should not be allowed to modify array elements.
 - C provides the type qualifier `const` (for “constant”) that can be used to prevent modification of array values in a function.
 - When an array parameter is preceded by the `const` qualifier, the array elements become constant in the function body, and any attempt to modify an element of the array in the function body results in a compile-time error.
 - This enables you to correct a program so it does not attempt to modify array elements.

Passing Arrays to Functions (cont'd)

- We now demonstrate the use of `const` qualifier.
- Function `tryToModifyArray` is defined with parameter `const int b[]`, which specifies that array `b` is constant and cannot be modified.
- The output shows the error messages produced by the compiler—the errors may be different for your compiler.
- Each of the function's three attempts to modify array elements results in the compiler error “l-value specifies a const object.” .



Software Engineering Observation 6.3

The `const` type qualifier can be applied to an array parameter in a function definition to prevent the original array from being modified in the function body. This is another example of the principle of least privilege. A function should not be given the capability to modify an array in the caller unless it's absolutely necessary.

Passing Arrays to Functions (cont'd)

```
1 // Using the const type qualifier with arrays.
2 #include <stdio.h>
3
4 void tryToModifyArray(const int b[]); // function prototype
5
6 // function main begins program execution
7 int main(void)
8 {
9     int a[] = { 10, 20, 30 }; // initialize array a
10
11     tryToModifyArray(a);
12
13     printf("%d %d %d\n", a[0], a[1], a[2]);
14
15     return 0;
16 }
17
18 // in function tryToModifyArray, array b is const, so it cannot be
19 // used to modify its argument array in the caller.
20 void tryToModifyArray(const int b[])
21 {
22     b[0] /= 2; // error
23     b[1] /= 2; // error
24     b[2] /= 2; // error
25 }
```

Sorting Arrays

- Sorting data
 - Placing the data into a particular order such as ascending or descending
 - One of the most important computing applications
- We will start with the simplest sorting algorithm.



Performance Tip 6.4

Often, the simplest algorithms perform poorly. Their virtue is that they're easy to write, test and debug. More complex algorithms are often needed to realize maximum performance.

Sorting Arrays

- Bubble Sort (sinking sort)
 - Based on comparisons
 - Make several passes through the array
 - ◆ The smaller values gradually “bubble” their way upward to the top of the array like air bubbles rising in water
 - ◆ The larger values sink to the bottom of the array
 - On each pass, successive pairs of elements (element 0 and element 1, then element 1 and element 2, etc.) are compared.
 - ◆ If a pair is in increasing order (or if the values are identical), we leave the values as they are.
 - ◆ If a pair is in decreasing order, their values are swapped in the array.
 - The array is sorted after several passes.

Sorting Arrays

- Pass 0: 2 6 4 8 10 12 89 68 45 37 
- Pass 1: 2 4 6 8 10 12 68 45 37 89
 - ◆ 2 < 6 unchanged: 2
 - ◆ 6 > 4 swapped: 2 4
 - ◆ 6 < 8 unchanged: 2 4 6
 - ◆ 8 < 10 unchanged: 2 4 6 8
 - ◆ 10 < 12 unchanged: 2 4 6 8 10
 - ◆ 12 < 89 unchanged: 2 4 6 8 10 12
 - ◆ 89 > 68 swapped: 2 4 6 8 10 12 68
 - ◆ 89 > 45 swapped: 2 4 6 8 10 12 68 45
 - ◆ 89 > 37 swapped: 2 4 6 8 10 12 68 45 37 89
- Pass 2: 2 4 6 8 10 12 45 37 68 89
- Pass 3: 2 4 6 8 10 12 37 45 68 89

Sorting Arrays

- Algorithm: BUBBLE-SORT (A)
Purpose: Sort the values in ascending order
Input: The set of integers
Output: Integers in ascending order
BUBBLE-SORT (A)
 - 1 **for** pass $\leftarrow 1$ **to** $|A| - 1$
 - 2 **do for** $i \leftarrow 1$ **to** $|A| - 1$
 - 3 **do if** $A_i > A_{i+1}$
 - 4 **then** swap (A_i, A_{i+1})
 - 5 **return** A

Sorting Arrays

```
1 // Sorting an array's values into ascending order.
2 #include <stdio.h>
3 #define SIZE 10
4
5 // function main begins program execution
6 int main(void)
7 {
8     // initialize a
9     int a[SIZE] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};
10
11     puts("Data items in original order");
12
13     // output original array
14     for (size_t i = 0; i < SIZE; ++i) {
15         printf("%4d", a[i]);
16     }
17
18     // bubble sort
19     // loop to control number of passes
20     for (unsigned int pass = 1; pass < SIZE; ++pass) {
21
22         // loop to control number of comparisons per pass
23         for (size_t i = 0; i < SIZE - 1; ++i) {
24
25             // compare adjacent elements and swap them if first
26             // element is greater than second element
27             if (a[i] > a[i + 1]) {
28                 int hold = a[i];
29                 a[i] = a[i + 1];
30                 a[i + 1] = hold;
31             }
32         }
33     }
34
35     puts("\nData items in ascending order");
36
37     // output sorted array
38     for (size_t i = 0; i < SIZE; ++i) {
39         printf("%4d", a[i]);
40     }
41
42     puts("");
43
44     return 0;
45 }
```

Data items in original order									
2	6	4	8	10	12	89	68	45	37
Data items in ascending order									
2	4	6	8	10	12	37	45	68	89

Sorting Arrays

- First the program compares $a[0]$ to $a[1]$, then $a[1]$ to $a[2]$, then $a[2]$ to $a[3]$, and so on until it completes the pass by comparing $a[8]$ to $a[9]$.
- Although there are 10 elements, only nine comparisons are performed.
- Because of the way the successive comparisons are made, a large value may move down the array many positions on a single pass, but a small value may move up only one position.
- On the first pass, the largest value is guaranteed to sink to the bottom element of the array, $a[9]$.

Sorting Arrays

- On the second pass, the second-largest value is guaranteed to sink to `a[8]`.
- On the ninth pass, the ninth-largest value sinks to `a[1]`.
- This leaves the smallest value in `a[0]`, so only nine passes of the array are needed to sort the array, even though there are ten elements.
- The sorting is performed by the nested for loops.

Sorting Arrays

- If a swap is necessary, it's performed by the three assignments

```
hold = a[i];  
a[i] = a[i + 1];  
a[i + 1] = hold;
```

where the extra variable hold temporarily stores one of the two values being swapped.

- The swap cannot be performed with only the two assignments

```
a[i] = a[i + 1];  
a[i + 1] = a[i];
```

- If, for example, $a[i]$ is 7 and $a[i + 1]$ is 5, after the first assignment both values will be 5 and the value 7 will be lost—hence the need for the extra variable hold.

Sorting Arrays

- The chief virtue of the bubble sort is that it's easy to program.
- However, it runs slowly because every exchange moves an element only one position closer to its final destination.
- This becomes apparent when sorting large arrays.

Sorting Arrays

- Making Improvement on Bubble Sort

- The largest value will sink to the bottom of the array
- There is no need to make comparisons with these sunk values.

BUBBLE-SORT (A)

```
1 for pass ← 1 to  $|A| - 1$ 
2   do for  $i \leftarrow 1$  to  $|A| - \text{pass} + 1$ 
3     do if  $A_i > A_{i+1}$ 
4       then swap ( $A_i, A_{i+1}$ )
5 return  $A$ 
```

Sorting Arrays

- Making Improvement on Bubble Sort

- The largest value will sink to the bottom of the array
- There is no need to make comparisons with these sunk values.

```
// bubble sort
// loop to control number of passes
for (unsigned int pass = 1; pass < SIZE; ++pass) {

    // loop to control number of comparisons per pass
    for (size_t i = 0; i < SIZE - pass; ++i) {

        // compare adjacent elements and swap them if first
        // element is greater than second element
        if (a[i] > a[i + 1]) {
            int hold = a[i];
            a[i] = a[i + 1];
            a[i + 1] = hold;
        }
    }
}
```

Sorting Arrays

- Further Improvement on Bubble Sort

- In each pass, no swap occurs implies all elements are in ascending order.
- In such case we can stop the comparison since the sorting has completed.

- BUBBLE-SORT (A)

```
1  for pass  $\leftarrow$  1 to  $|A| - 1$ 
2      swapped  $\leftarrow$  0
3      do for  $i \leftarrow 1$  to  $|A| - \text{pass} + 1$ 
4          do if  $A_i > A_{i+1}$ 
5              then swap ( $A_i, A_{i+1}$ )
6                  swapped  $\leftarrow$  1
7      if swapped
8          then return  $A$ 
9  return  $A$ 
```

Case Study: Computing Mean, Median and Mode Using Arrays

- Computers are commonly used for **survey data analysis** to compile and analyze the results of surveys and opinion polls.
- For example, we can use array to store responses to a survey:
 - 99 responses at initialization
 - Each response is a number from 1 to 9.
- We will show the program computing the **mean, median and mode** of the 99 values.
- This example includes most of the common manipulations usually required in array problems, including passing arrays to functions.

Case Study: Computing Mean, Median and Mode Using Arrays

```
1 // Survey data analysis with arrays;
2 // computing the mean, median and mode of the data.
3 #include <stdio.h>
4 #define SIZE 99
5
6 // function prototypes
7 void mean(const unsigned int answer[]);
8 void median(unsigned int answer[]);
9 void mode(unsigned int freq[], const unsigned int answer[]) ;
10 void bubbleSort(unsigned int a[]);
11 void printArray(const unsigned int a[]);
12
13 // function main begins program execution
14 int main(void)
15 {
16     unsigned int frequency[10] = {0}; // initialize array frequency
17
18     // initialize array response
19     unsigned int response[SIZE] =
20     {
21         6, 7, 8, 9, 8, 7, 8, 9, 8, 9,
22         7, 8, 9, 5, 9, 8, 7, 8, 7, 8,
23         6, 7, 8, 9, 3, 9, 8, 7, 8, 7,
24         7, 8, 9, 8, 9, 8, 9, 7, 8, 9,
25         6, 7, 8, 7, 8, 7, 9, 8, 9, 2,
26         7, 8, 9, 8, 9, 8, 9, 7, 5, 3,
27         5, 6, 7, 2, 5, 3, 9, 4, 6, 4,
28         7, 8, 9, 6, 8, 7, 8, 9, 7, 8,
29         7, 4, 4, 2, 5, 3, 8, 7, 5, 6,
30         4, 5, 6, 1, 6, 5, 7, 8, 7};
31
32     // process responses
33     mean(response);
34     median(response);
35     mode(frequency, response);
36
37     return 0;
38 }
```

Case Study: Computing Mean, Median and Mode Using Arrays

```
39 // calculate average of all response values
40 void mean(const unsigned int answer[])
41 {
42     printf("%s\n%s\n%s\n", "*****", "   Mean", "*****");
43
44     unsigned int total = 0; // variable to hold sum of array elements
45
46     // total response values
47     for (size_t j = 0; j < SIZE; ++j) {
48         total += answer[j];
49     }
50
51     printf("The mean is the average value of the data\n"
52           "items. The mean is equal to the total of\n"
53           "all the data items divided by the number\n"
54           "of data items (%u). The mean value for\n"
55           "this run is: %u / %u = %.4f\n\n",
56           SIZE, total, SIZE, (double) total / SIZE);
57 }
```

Case Study: Computing Mean, Median and Mode Using Arrays

```
59 // sort array and determine median element's value
60 void median(unsigned int answer[])
61 {
62     printf("\n%s\n%s\n%s\n%s",
63           "*****", " Median", "*****",
64           "The unsorted array of responses is");
65
66     printArray(answer); // output unsorted array
67
68     bubbleSort(answer); // sort array
69
70     printf("%s", "\n\nThe sorted array is");
71     printArray(answer); // output sorted array
72
73     // display median element
74     printf("\n\nThe median is element %u of\n"
75           "the sorted %u element array.\n"
76           "For this run the median is %u\n\n",
77           SIZE / 2, SIZE, answer[SIZE / 2]);
78 }
```

Case Media

an,
ays

```
80 // determine most frequent response
81 void mode(unsigned int freq[], const unsigned int answer[])
82 {
83     printf("\n%s\n%s\n%s\n", "*****", " Mode", "*****");
84
85     // initialize frequencies to 0
86     for (size_t rating = 1; rating <= 9; ++rating) {
87         freq[rating] = 0;
88     }
89
90     // summarize frequencies
91     for (size_t j = 0; j < SIZE; ++j) {
92         ++freq[answer[j]];
93     }
94
95     // output headers for result columns
96     printf("%s%11s%19s\n\n%54s\n%54s\n\n",
97           "Response", "Frequency", "Histogram",
98           "1    1    2    2", "5    0    5    0    5");
99
100    // output results
101    unsigned int largest = 0; // represents largest frequency
102    unsigned int modeValue = 0; // represents most frequent response
103
104    for (size_t rating = 1; rating <= 9; ++rating) {
105        printf("%8u%11u", rating, freq[rating]);
106
107        // keep track of mode value and largest frequency value
108        if (freq[rating] > largest) {
109            largest = freq[rating];
110            modeValue = rating;
111        }
112
113        // output histogram bar representing frequency value
114        for (unsigned int h = 1; h <= freq[rating]; ++h) {
115            printf("%s", "*");
116        }
117
118        puts(""); // being new line of output
119    }
120
121    // display the mode value
122    printf("\nThe mode is the most frequent value.\n"
123          "For this run the mode is %u which occurred"
124          " %u times.\n", modeValue, largest);
125 }
```

Case Study: Computing Mean, Median and Mode Using Arrays

```
127 // function that sorts an array with bubble sort algorithm
128 void bubbleSort(unsigned int a[])
129 {
130     // loop to control number of passes
131     for (unsigned int pass = 1; pass < SIZE; ++pass) {
132
133         // loop to control number of comparisons per pass
134         for (size_t j = 0; j < SIZE - 1; ++j) {
135
136             // swap elements if out of order
137             if (a[j] > a[j + 1]) {
138                 unsigned int hold = a[j];
139                 a[j] = a[j + 1];
140                 a[j + 1] = hold;
141             }
142         }
143     }
144 }
145
146 // output array contents (20 values per row)
147 void printArray(const unsigned int a[])
148 {
149     // output array contents
150     for (size_t j = 0; j < SIZE; ++j) {
151
152         if (j % 20 == 0) { // begin new line every 20 values
153             puts("");
154         }
155
156         printf("%2u", a[j]);
157     }
158 }
```

Case Study: Computing Mean, Median and Mode Using Arrays

The mean is the average value of the data items. The mean is equal to the total of all the data items divided by the number of data items (99). The mean value for this run is: $681 / 99 = 6.8788$

The median is element 49 of the sorted 99 element array. For this run the median is 7

```
*  
**  
***  
****  
*****  
******  
*****  
****  
***  
**  
*
```

The mode is the most frequent value.
For this run the mode is 8 which occurred 27 times.

Case Study: Computing Mean, Median and Mode Using Arrays

- Mean

- The mean is the arithmetic average of the 99 values.
- Function mean computes the mean by totaling the 99 elements and dividing the result by 99.

Case Study: Computing Mean, Median and Mode Using Arrays (cont'd)

- Median

- The median is the “middle value.”
- Function `median` determines the median by calling function `bubbleSort` to sort the array of responses into ascending order, then picking `answer[SIZE / 2]` (the middle element) of the sorted array.
- When the number of elements is even, the median should be calculated as the mean of the two middle elements.
- Function `median` does not currently provide this capability.
- Function `printArray` is called to output the response array.

Case Study: Computing Mean, Median and Mode Using Arrays.

- Mode

- The mode is the value that occurs most frequently among the 99 responses.
- Function `mode` determines the mode by counting the number of responses of each type, then selecting the value with the greatest count.
- This version of function `mode` does not handle a tie
- Function `mode` also produces a histogram to aid in determining the mode graphically.

Searching Arrays

- It may be necessary to determine whether an array contains a value that matches a certain key value.
- The process of finding a particular element of an array is called searching.
- In this section we discuss two searching techniques—the simple **linear search** technique and the more efficient (but more complex) **binary search** technique.

Searching Arrays (cont'd)

- Searching an Array with Linear Search
 - The linear search compares each element of the array with the search key.
 - Because the array is not in any particular order, it's just as likely that the value will be found in the first element as in the last.
 - On average, therefore, the program will have to compare the search key with half the elements of the array.

Searching Arrays (cont'd)

- Algorithm: LINEAR-SEARCH (A, key)

Purpose: Find and return the position of key in A

Input: A set of integers, key value

Output: the position

LINEAR-SEARCH (A, key)

```
1 for  $i \leftarrow 1$  to  $|A|$ 
2   do if  $A_i = key$ 
3     then return  $i$ 
4 return not found
```

Searching Arrays (cont'd)

```
1 // Linear search of an array.
2 #include <stdio.h>
3 #define SIZE 100
4
5 // function prototype
6 size_t linearSearch(const int array[], int key, size_t size);
7
8 // function main begins program execution
9 int main(void)
10 {
11     int a[SIZE]; // create array a
12
13     // create some data
14     for (size_t x = 0; x < SIZE; ++x) {
15         a[x] = 2 * x;
16     }
17
18     printf("Enter integer search key: ");
19     int searchKey; // value to locate in array a
20     scanf("%d", &searchKey);
21
22     // attempt to locate searchKey in array a
23     size_t index = linearSearch(a, searchKey, SIZE);
24
25     // display results
26     if (index != -1) {
27         printf("Found value at index %d\n", index);
28     }
29     else {
30         puts("Value not found");
31     }
32
33     return 0;
34 }
```

```
36 // compare key to every element of array until the location is found
37 // or until the end of array is reached; return index of element
38 // if key is found or -1 if key is not found
39 size_t linearSearch(const int array[], int key, size_t size)
40 {
41     // loop through array
42     for (size_t n = 0; n < size; ++n) {
43
44         if (array[n] == key) {
45             return n; // return location of key
46         }
47     }
48
49     return -1; // key not found
50 }
```

Enter integer search key: 36
Found value at index 18

Enter integer search key: 37
Value not found

Searching Arrays (cont'd)

- Searching an Array with Binary Search
 - The linear searching method works well for small or unsorted arrays.
 - However, for large arrays linear searching is inefficient.
 - If the array is sorted, the high-speed binary search technique can be used.
 - The binary search algorithm **eliminates** from consideration **one-half** of the elements in a sorted array after each comparison.
 - The algorithm **locates the middle element** of the array and compares it to the search key.

Searching Arrays (cont'd)

- If they're equal, the search key is found and the index of that element is returned.
- If they're not equal, the problem is reduced to searching one-half of the array.
- If the search key is **less** than the middle element of the array, the **first half** of the array is searched, otherwise the second half of the array is searched.
- If the search key is not found in the specified subarray (piece of the original array), the algorithm is repeated on one-quarter of the original array.

Searching Arrays (cont'd)

- The search continues until the search key is equal to the middle element of a subarray, or until the subarray consists of one element that is not equal to the search key (i.e., the search key is not found).
- In a worst case-scenario, searching an array of 1023 elements takes only 10 comparisons using a binary search.
- Repeatedly dividing 1,024 by 2 yields the values 512, 256, 128, 64, 32, 16, 8, 4, 2 and 1.
- The number 1,024 (2^{10}) is divided by 2 only 10 times to get the value 1.
- Dividing by 2 is equivalent to one comparison in the binary search algorithm.

Searching Arrays (cont'd)

- An array of 1,048,575 (2^{20}) elements takes a maximum of only 20 comparisons to find the search key.
- An array of one billion elements takes a maximum of only 30 comparisons to find the search key.
- This is a tremendous increase in performance over the linear search that required comparing the search key to an average of half of the array elements.
- For a one-billion-element array, this is a difference between an average of 500 million comparisons and a maximum of 30 comparisons!

Searching Arrays (cont'd)

- The **maximum comparisons** for any array can be determined by finding **the first power of 2 greater than the number** of array elements.
- Next example presents the iterative version of function `binarySearch`.
- The function receives four arguments—an integer array `b` to be searched, an integer `searchKey`, the `low` array index and the `high` array index (these define the portion of the array to be searched).
- If the search key does not match the middle element of a subarray, the `low` index or `high` index is modified so that a smaller subarray can be searched.

Searching Arrays (cont'd)

- If the search key is less than the middle element, the `high` index is set to `middle - 1` and the search is continued on the elements from `low` to `middle - 1`.
- If the search key is greater than the middle element, the `low` index is set to `middle + 1` and the search is continued on the elements from `middle + 1` to `high`.
- The program uses an array of 15 elements.
- The first power of 2 greater than the number of elements in this array is 16 (2^4), so no more than 4 comparisons are required to find the search key.

Searching Arrays (cont'd)

- The program uses function `printHeader` to output the array indices and function `printRow` to output each subarray during the binary search process.
- The middle element in each subarray is marked with an asterisk (*) to indicate the element to which the search key is compared.

Searching Arrays (cont'd)

- Algorithm: BINARY-SEARCH (A, key)

Purpose: Find and return the position of key in A

Input: A set of integers, key value

Output: The position

BINARY-SEARCH (A, key)

1 $low \leftarrow 1, high \leftarrow |L|$

2 **while** $high > low$

3 **do** $mid \leftarrow (low + high) / 2$

4 **if** $A_{mid} = key$

5 **then** return mid

6 **if** $A_{mid} > key$

7 **then** $high = mid - 1$

8 **if** $A_{mid} < key$

9 **then** $low = mid + 1$

10 **return** not found

Searching Arrays (cont'd)

```
1 // Binary search of a sorted array.
2 #include <stdio.h>
3 #define SIZE 15
4
5 // function prototypes
6 size_t binarySearch(const int b[], int searchKey, size_t low, size_t high);
7 void printHeader(void);
8 void printRow(const int b[], size_t low, size_t mid, size_t high);
9
10 // function main begins program execution
11 int main(void)
12 {
13     int a[SIZE]; // create array a
14
15     // create data
16     for (size_t i = 0; i < SIZE; ++i) {
17         a[i] = 2 * i;
18     }
19
20     printf("%s", "Enter a number between 0 and 28: ");
21     int key; // value to locate in array a
22     scanf("%d", &key);
23
24     printHeader();
25
26     // search for key in array a
27     size_t result = binarySearch(a, key, 0, SIZE - 1);
28
29     // display results
30     if (result != -1) {
31         printf("\n%d found at index %d\n", key, result);
32     }
33     else {
34         printf("\n%d not found\n", key);
35     }
36
37     return 0;
38 }
```

Searching Arrays (cont'd)

```
40 // function to perform binary search of an array
41 size_t binarySearch(const int b[], int searchKey, size_t low, size_t high)
42 {
43     // loop until low index is greater than high index
44     while (low <= high) {
45
46         // determine middle element of subarray being searched
47         size_t middle = (low + high) / 2;
48
49         // display subarray used in this loop iteration
50         printRow(b, low, middle, high);
51
52         // if searchKey matched middle element, return middle
53         if (searchKey == b[middle]) {
54             return middle;
55         }
56
57         // if searchKey is less than middle element, set new high
58         else if (searchKey < b[middle]) {
59             high = middle - 1; // search low end of array
60         }
61
62         // if searchKey is greater than middle element, set new low
63         else {
64             low = middle + 1; // search high end of array
65         }
66     } // end while
67
68     return -1; // searchKey not found
69 }
```

Searching Arrays (cont'd)

```
71 // Print a header for the output
72 void printHeader(void)
73 {
74     puts("\Indices:");
75
76     // output column head
77     for (unsigned int i = 0; i < SIZE; ++i) {
78         printf("%3u ", i);
79     }
80
81     puts(""); // start new line of output
82
83     // output line of - characters
84     for (unsigned int i = 1; i <= 4 * SIZE; ++i) {
85         printf("%s", "-");
86     }
87
88     puts(""); // start new line of output
89 }
```

Searching Arrays (cont'd)

```
91 // Print one row of output showing the current
92 // part of the array being processed.
93 void printRow(const int b[], size_t low, size_t mid, size_t high)
94 {
95     // loop through entire array
96     for (size_t i = 0; i < SIZE; ++i) {
97
98         // display spaces if outside current subarray range
99         if (i < low || i > high) {
100             printf("%s", "    ");
101         }
102         else if (i == mid) { // display middle element
103             printf("%3d*", b[i]); // mark middle value
104         }
105         else { // display other elements in subarray
106             printf("%3d ", b[i]);
107         }
108     }
109
110     puts(""); // start new line of output
111 }
```

Searching Arrays (cont'd)

Enter a number between 0 and 28: 25

Indices:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

0	2	4	6	8	10	12	14*	16	18	20	22	24	26	28
								16	18	20	22*	24	26	28
												24	26*	28
												24*		

25 not found

Enter a number between 0 and 28: 8

Indices:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

0	2	4	6	8	10	12	14*	16	18	20	22	24	26	28
0	2	4	6*	8	10	12								
				8	10*	12								
				8*										

8 found at index 4

Enter a number between 0 and 28: 6

Indices:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

0	2	4	6	8	10	12	14*	16	18	20	22	24	26	28
0	2	4	6*	8	10	12								

6 found at index 3

Multidimensional Arrays

- Arrays in C can have multiple indices.
- A common use of multidimensional arrays is to represent tables of values consisting of information arranged in rows and columns.
- To identify a particular table element, we must specify two indices:
 - The first (by convention) identifies the element's row
 - The second (by convention) identifies the element's column.
- Tables or arrays that require two indices to identify a particular element are called two-dimensional arrays.

Multidimensional Arrays (cont'd)

- Multidimensional arrays can have more than two indices.
- We will presents an example of a two-dimensional array, a, with three rows and four columns, so it's said to be a 3-by-4 array.
- In general, an array with m rows and n columns is called an m-by-n array

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Column index
Row index
Array name

Multidimensional Arrays (cont'd)

- Every element in array a is identified by an element name of the form $a[i][j]$; a is the name of the array, and i and j are the indices that uniquely identify each element in a .
- The names of the elements in row 0 all have a first index of 0; the names of the elements in column 3 all have a second index of 3.



Common Programming Error 6.7

Referencing a two-dimensional array element as $a[x, y]$ instead of $a[x][y]$ is a logic error. C interprets $a[x, y]$ as $a[y]$ (because the comma in this context is treated as a comma operator), so this programmer error is not a syntax error.

Multidimensional Arrays (cont'd)

- A multidimensional array can be initialized when it's defined, much like a one-dimensional array.
- For example, a two-dimensional array `int b[2][2]` could be defined and initialized with

```
int b[2][2] = {{1, 2}, {3, 4}};
```

- The values are **grouped by row in braces**.
 - The values in the first set of braces initialize row 0 and the values in the second set of braces initialize row 1.
 - So, the values 1 and 2 initialize elements `b[0][0]` and `b[0][1]`, respectively, and the values 3 and 4 initialize elements `b[1][0]` and `b[1][1]`, respectively.

Multidimensional Arrays (cont'd)

- If there are not enough initializers for a given row, the **remaining** elements of that row are initialized to **0**. Thus,

```
int b[2][2] = {{1}, {3, 4}};
```

would initialize `b[0][0]` to 1, `b[0][1]` to 0, `b[1][0]` to 3 and `b[1][1]` to 4.

- Next we show how to define and initialize two-dimensional arrays.

Multidimensional Arrays (cont'd)

```
24 // function to output array with two rows and three columns
25 void printArray(int a[][3])
26 {
27     // loop through rows
28     for (size_t i = 0; i <= 1; ++i) {
29
30         // output column values
31         for (size_t j = 0; j <= 2; ++j) {
32             printf("%d ", a[i][j]);
33         }
34
35         printf("\n"); // start new line of output
36     }
37 }
```

Multidimensional Arrays (cont'd)

```
1 // Initializing multidimensional arrays.
2 #include <stdio.h>
3
4 void printArray(int a[][3]); // function prototype
5
6 // function main begins program execution
7 int main(void)
8 {
9     int array1[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };
10    puts("Values in array1 by row are:");
11    printArray(array1);
12
13    int array2[2][3] = { 1, 2, 3, 4, 5 };
14    puts("Values in array2 by row are:");
15    printArray(array2);
16
17    int array3[2][3] = { { 1, 2 }, { 4 } };
18    puts("Values in array3 by row are:");
19    printArray(array3);
20
21    return 0;
22 }
```

Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0

4 0 0
1 5 0

Multidimensional Arrays (cont'd)

- Example of Two-dimensional Arrays

- The program defines three arrays of two rows and three columns (six elements each).
- The definition of array1 provides six initializers in two sublists.
 - ◆ The first sublist initializes row 0 of the array to the values 1, 2 and 3
 - ◆ The second sublist initializes row 1 of the array to the values 4, 5 and 6.

Multidimensional Arrays (cont'd)

- If the braces around each sublist are removed from the `array1` initializer list, the compiler initializes the elements of the first row followed by the elements of the second row.
- The definition of `array2` provides five initializers.
 - ◆ The initializers are assigned to the first row, then the second row.
 - ◆ Any elements that do not have an explicit initializer are initialized to zero automatically, so `array2[1][2]` is initialized to 0.

Multidimensional Arrays (cont'd)

- The definition of `array3` provides three initializers in two sublists.
 - ◆ The sublist for the first row explicitly initializes the first two elements of the first row to 1 and 2.
 - ◆ The third element is initialized to zero.
 - ◆ The sublist for the second row explicitly initializes the first element to 4.
 - ◆ The last two elements are initialized to zero.
- The program calls `printArray` to output each array's elements.
 - ◆ The function definition specifies the array parameter as `const int a[][3]`.
 - ◆ When we receive a one-dimensional array as a parameter, the array brackets are empty in the function's parameter list.

Multidimensional Arrays (cont'd)

- ◆ The first index of a multidimensional array is not required either, but **all subsequent indices are required**.
- ◆ The compiler uses these indices to determine the locations in memory of elements in multidimensional arrays.
- ◆ All array elements are stored consecutively in memory regardless of the number of indices.
- ◆ In a two-dimensional array, the first row is stored in memory followed by the second row.
- ◆ Providing the index values in a parameter declaration enables the compiler to tell the function how to locate an element in the array.

Multidimensional Arrays (cont'd)

- In a two-dimensional array, each row is basically a one-dimensional array.
- To locate an element in a particular row, the compiler must know how many elements are in each row so that it can skip the proper number of memory locations when accessing the array.
- Thus, when accessing `a[1][2]` in our example, the compiler knows to skip the three elements of the first row to get to the second row (row 1).
- Then, the compiler accesses element 2 of that row.

Multidimensional Arrays (cont'd)

- Many common array manipulations use for iteration statements.
- For example, the following statement sets all the elements in row 2 of array a to zero:

```
for (size_t column = 0; column <= 3; ++column) {  
    a[2][column] = 0;  
}
```

We specified row 2, so the first index is always 2, and the loop varies only the second index.

- The preceding for statement is equivalent to the assignment statements:

```
a[2][0] = 0;  
a[2][1] = 0;  
a[2][2] = 0;  
a[2][3] = 0;
```

Multidimensional Arrays (cont'd)

- The following nested for statement determines the total of all the elements in array a.

```
total = 0;
for (size_t row = 0; row <= 2; ++row) {
    for (size_t column = 0; column <= 3; ++column) {
        total += a[row][column];
    }
}
```

- The for statement totals the elements of the array one row at a time.

Multidimensional Arrays (cont'd)

- The outer for statement begins by setting row (i.e., the row index) to 0 so that the elements of that row may be totaled by the inner for statement.
- The outer for statement then increments row to 1, so the elements of that row can be totaled.
- Then, the outer for statement increments row to 2, so the elements of the third row can be totaled.
- When the nested for statement terminates, total contains the sum of all the elements in the array a.

Multidimensional Arrays (cont'd)

- Two-Dimensional Array Manipulations
 - Next example performs several other common array manipulations on 3-by-4 array `studentGrades` using for statements.
 - Each row of the array represents a student and each column represents a grade on one of the four exams the students took during the semester.
 - The array manipulations are performed by four functions.
 - Function `minimum` determines the lowest grade of any student for the semester.

Multidimensional Arrays (cont'd)

- Function `maximum` determines the highest grade of any student for the semester.
- Function `average` determines a particular student's semester average.
- Function `printArray` outputs the two-dimensional array in a neat, tabular format.

Multidimensional Arrays

(co

```
1 // Two-dimensional array manipulations.
2 #include <stdio.h>
3 #define STUDENTS 3
4 #define EXAMS 4
5
6 // function prototypes
7 int minimum(const int grades[][EXAMS], size_t pupils, size_t tests);
8 int maximum(const int grades[][EXAMS], size_t pupils, size_t tests);
9 double average(const int setOfGrades[], size_t tests);
10 void printArray(const int grades[][EXAMS], size_t pupils, size_t tests);
11
12 // function main begins program execution
13 int main(void)
14 {
15     // initialize student grades for three students (rows)
16     int studentGrades[STUDENTS][EXAMS] =
17     { { 77, 68, 86, 73 },
18       { 96, 87, 89, 78 },
19       { 70, 90, 86, 81 } };
20
21     // output array studentGrades
22     puts("The array is:");
23     printArray(studentGrades, STUDENTS, EXAMS);
24
25     // determine smallest and largest grade values
26     printf("\n\nLowest grade: %d\nHighest grade: %d\n",
27           minimum(studentGrades, STUDENTS, EXAMS),
28           maximum(studentGrades, STUDENTS, EXAMS));
29
30     // calculate average grade for each student
31     for (size_t student = 0; student < STUDENTS; ++student) {
32         printf("The average grade for student %u is %.2f\n",
33               student, average(studentGrades[student], EXAMS));
34     }
35
36     return 0;
37 }
```

Multidimensional Arrays

(con

```
39 // Find the minimum grade
40 int minimum(const int grades[][EXAMS], size_t pupils, size_t tests)
41 {
42     int lowGrade = 100; // initialize to highest possible grade
43
44     // loop through rows of grades
45     for (size_t i = 0; i < pupils; ++i) {
46
47         // loop through columns of grades
48         for (size_t j = 0; j < tests; ++j) {
49
50             if (grades[i][j] < lowGrade) {
51                 lowGrade = grades[i][j];
52             }
53         }
54     }
55
56     return lowGrade; // return minimum grade
57 }
58
59 // Find the maximum grade
60 int maximum(const int grades[][EXAMS], size_t pupils, size_t tests)
61 {
62     int highGrade = 0; // initialize to lowest possible grade
63
64     // loop through rows of grades
65     for (size_t i = 0; i < pupils; ++i) {
66
67         // loop through columns of grades
68         for (size_t j = 0; j < tests; ++j) {
69
70             if (grades[i][j] > highGrade) {
71                 highGrade = grades[i][j];
72             }
73         }
74     }
75
76     return highGrade; // return maximum grade
77 }
```

Multidimensional Arrays (cont'd)

```
79 // Determine the average grade for a particular student
80 double average(const int setOfGrades[], size_t tests)
81 {
82     int total = 0; // sum of test grades
83
84     // total all grades for one student
85     for (size_t i = 0; i < tests; ++i) {
86         total += setOfGrades[i];
87     }
88
89     return (double) total / tests; // average
90 }
91
92 // Print the array
93 void printArray(const int grades[][EXAMS], size_t pupils, size_t tests)
94 {
95     // output column heads
96     printf("%s", "          [0]  [1]  [2]  [3]");
97
98     // output grades in tabular format
99     for (size_t i = 0; i < pupils; ++i) {
100
101         // output label for row
102         printf("\nstudentGrades[%u] ", i);
103
104         // output grades for one student
105         for (size_t j = 0; j < tests; ++j) {
106             printf("%-5d", grades[i][j]);
107         }
108     }
109 }
110
```

The array is:

	[0]	[1]	[2]	[3]
studentGrades[0]	77	68	86	73
studentGrades[1]	96	87	89	78
studentGrades[2]	70	90	86	81

Lowest grade: 68

Highest grade: 96

The average grade for student 0 is 76.00

The average grade for student 1 is 87.50

The average grade for student 2 is 81.75

Multidimensional Arrays (cont'd)

- Functions `minimum`, `maximum` and `printArray` each receive three arguments—the `studentGrades` array (called `grades` in each function), the number of students (rows of the array) and the number of exams (columns of the array).
- Each function loops through array grades using nested for statements.

Multidimensional Arrays (cont'd)

- The following nested for statement is from the function minimum definition:

```
■ // loop through rows of grades
  for (size_t i = 0; i < pupils; ++i) {
    // loop through columns of grades
    for (size_t j = 0; j < tests; ++j) {
      if (grades[i][j] < lowGrade) {
        lowGrade = grades[i][j];
      }
    }
  }
```

Multidimensional Arrays (cont'd)

- The outer for statement begins by setting `i` (i.e., the row index) to 0 so that the elements of that row (i.e., the grades of the first student) can be compared to variable `lowGrade` in the body of the inner for statement.
- The inner for statement loops through the four grades of a particular row and compares each grade to `lowGrade`.
- If a grade is less than `lowGrade`, `lowGrade` is set to that grade.
- The outer for statement then increments the row index to 1.
- The elements of that row are compared to variable `lowGrade`.

Multidimensional Arrays (cont'd)

- The outer for statement then increments the row index to 2.
- The elements of that row are compared to variable `lowGrade`.
- When execution of the nested statement is complete, `lowGrade` contains the smallest grade in the two-dimensional array.
- Function `maximum` works similarly to function `minimum`.
- Function `average` takes two arguments—a one-dimensional array of test results for a particular student called `setOfGrades` and the number of test results in the array.

Multidimensional Arrays.

- When `average` is called, the first argument `studentGrades[student]` is passed.
- This causes the address of one row of the two-dimensional array to be passed to `average`.
- The argument `studentGrades[1]` is the starting address of row 1 of the array.
- Remember that a two-dimensional array is basically an array of one-dimensional arrays and that the name of a one-dimensional array is the address of the array in memory.
- Function `average` calculates the sum of the array elements, divides the total by the number of test results and returns the floating-point result.

Variable-Length Arrays

- In early versions of C, all arrays had constant size.
- But what if you don't know an array's size at compilation time?
- To handle this, you'd have to use **dynamic memory allocation with `malloc`** and related functions.
- The C standard allows you to handle arrays of unknown size using **variable-length arrays (VLAs)**.
- These are not arrays whose size can change—that would compromise the integrity of nearby locations in memory.

Variable-Length Arrays (cont'd)

- A variable-length array is an array whose length, or size, is defined in terms of an expression evaluated at execution time.
- The following program declares and prints several VLAs.

Note: This feature is not supported in Microsoft Visual C++.

Variable-Length Arrays (cont'd)

```
1 // Using variable-length arrays in C99
2 #include <stdio.h>
3
4 // function prototypes
5 void print1DArray(size_t size, int array[size]);
6 void print2DArray(size_t row, size_t col, int array[row][col]);
7
8 int main(void)
9 {
10     printf("%s", "Enter size of a one-dimensional array: ");
11     int arraySize; // size of 1-D array
12     scanf("%d", &arraySize);
13
14     int array[arraySize]; // declare 1-D variable-length array
15
16     printf("%s", "Enter number of rows and columns in a 2-D array: ");
17     int row1, col1; // number of rows and columns in a 2-D array
18     scanf("%d %d", &row1, &col1);
19
20     int array2D1[row1][col1]; // declare 2-D variable-length array
21
22     printf("%s",
23         "Enter number of rows and columns in another 2-D array: ");
24     int row2, col2; // number of rows and columns in another 2-D array
25     scanf("%d %d", &row2, &col2);
26
27     int array2D2[row2][col2]; // declare 2-D variable-length array
28
29     // test sizeof operator on VLA
30     printf("\nsizeof(array) yields array size of %d bytes\n",
31         sizeof(array));
32 }
```

Variable-Length Arrays (cont'd)

```
33 // assign elements of 1-D VLA
34 for (size_t i = 0; i < arraySize; ++i) {
35     array[i] = i * i;
36 }
37
38 // assign elements of first 2-D VLA
39 for (size_t i = 0; i < row1; ++i) {
40     for (size_t j = 0; j < col1; ++j) {
41         array2D1[i][j] = i + j;
42     }
43 }
44
45 // assign elements of second 2-D VLA
46 for (size_t i = 0; i < row2; ++i) {
47     for (size_t j = 0; j < col2; ++j) {
48         array2D2[i][j] = i + j;
49     }
50 }
51
52 puts("\nOne-dimensional array:");
53 print1DArray(arraySize, array); // pass 1-D VLA to function
54
55 puts("\nFirst two-dimensional array:");
56 print2DArray(row1, col1, array2D1); // pass 2-D VLA to function
57
58 puts("\nSecond two-dimensional array:");
59 print2DArray(row2, col2, array2D2); // pass other 2-D VLA to function
60
61 return 0;
62 }
```

Variable-Length Arrays (cont'd)

```
64 void print1DArray(size_t size, int array[size])
65 {
66     // output contents of array
67     for (size_t i = 0; i < size; i++) {
68         printf("array[%d] = %d\n", i, array[i]);
69     }
70 }
71
72 void print2DArray(size_t row, size_t col, int array[row][col])
73 {
74     // output contents of array
75     for (size_t i = 0; i < row; ++i) {
76         for (size_t j = 0; j < col; ++j) {
77             printf("%5d", array[i][j]);
78         }
79         puts("");
80     }
81 }
82 }
```

Enter size of a one-dimensional array: 6
Enter number of rows and columns in a 2-D array: 2 5
Enter number of rows and columns in another 2-D array: 4 3

sizeof(array) yields array size of 24 bytes

One-dimensional array:

array[0] = 0
array[1] = 1
array[2] = 4
array[3] = 9
array[4] = 16
array[5] = 25

First two-dimensional array:

0	1	2	3	4
1	2	3	4	5

Second two-dimensional array:

0	1	2
1	2	3
2	3	4
3	4	5

Variable-Length Arrays (cont'd)

- First, we prompt the user for the desired sizes for a one-dimensional array and two two-dimensional arrays .
- Next we declare VLAs of the appropriate size.
- This is valid as long as the variables representing the `array sizes are of an integral type`.
- After declaring the arrays, we use the `sizeof` operator to make sure that our VLA is of the proper length.
- In early versions of C `sizeof` was always a compile-time operation, `but when applied to a VLA, sizeof operates at runtime`.

Variable-Length Arrays (cont'd)

`sizeof(array)` yields array size of 24 bytes

- The output window shows that the `sizeof` operator returns a size of 24 bytes—four times that of the number we entered because the size of an `int` on our machine is 4 bytes.
- Next we assign values to the elements of our VLAs.
- We use `i < arraySize` as our loop-continuation condition when filling the one-dimensional array.
- As with fixed-length arrays, there is no protection against stepping outside the array bounds.
- Function `print1DArray` takes a one-dimensional VLA.

Variable-Length Arrays (cont'd)

- The syntax for passing VLAs as parameters to functions is the same as with a normal, fixed-length array.
- We use the variable size in the declaration of the array parameter, but no checking is performed other than the variable being defined and of integral type—it's purely documentation for the programmer.
- Function `print2DArray` takes a variable-length two-dimensional array and displays it to the screen.
- Recall that, **all but the first index of a multidimensional array must be specified** when declaring a function parameter.

Variable-Length Arrays.

- The same restriction holds true for VLAs, except that the sizes can be specified by variables.
- The initial value of `col` passed to the function is used to convert from two-dimensional indices to offsets into the contiguous memory the array is stored in, just as with a fixed-size array.
- Changing the value of `col` inside the function will not cause any changes to the indexing, but passing an incorrect value to the function will.

Exercises

- Briefly describe the definition of array in C.
- What will happen for the following two definitions?
 - a) `int array[5] = {0, 1, 2, 3}`
 - b) `int array[5] = {0, 1, 2, 3, 4, 5};`
- List the array contents of the following definition and describe how it happened.
`int array[5] = {1};`

Exercises

- What will happen before compilation when symbolic constant is used?
- Give two advantages of using symbolic constant.
- What will happen when using printf function to print a string without null character?

Exercises

- Given the following function, show the array contents after calling it twice.

```
void staticArray()  
{  
    static int array[5];  
    for (size_t i = 0, i < 5, ++i) {  
        array[i] += 3;  
    }  
}
```

- When should we use linear search and binary search?

Exercises

- Show the contents of the following arrays by tables.

```
int array1[2][3] =  
{ {1, 2, 3}, {4, 5, 6} };  
int array2[2][3] = {1, 2, 3, 4, 5};  
int array3[2][3] = { {1, 2}, {4} };
```

- Give the two representations of the solution of the n-Queen puzzle. How to use them to represent the following solution of 4-Queen puzzle?

Exercises

- What is variable length array in C? What is the difference when applying `sizeof` to constant size array and variable length array?
- Give a brief description of buffer overflow.

Exercises

- Show how to define an array of integer of size 10 with all zero at initialization using symbolic constant `SIZE`.
- How to prevent the `scanf` from inputting too many characters which surpassing the array length? Use the following definition as an example to show your answer.
`char string[100];`

Exercises

- A vector is a sequence of numbers which can be represented by an array in C. The addition of two vectors is defined as follows. Given two vectors U and V of the same size, the i^{th} element of vector $W=U+V$ has the value $w_i = u_i + v_i$. Give a function named as `vecAdd` for the vector addition $W=U+V$ of any size. Assume that the arrays are of integer type.

Exercises

- Show how to prevent the calling function from modifying the contents of an array argument. You have to give the function prototype to show your answer.
- Write the bubble sort.

Exercises

- Write the insertion sort according to the following algorithm.

```
1  for  $i \leftarrow 2$  to  $|A|$ 
2      do  $key \leftarrow A_i$ 
3           $j \leftarrow i - 1$ 
4          while  $j > 0$  and  $A_j > key$ 
5              do  $A_{j+1} \leftarrow A_j$ 
6                   $j \leftarrow j - 1$ 
7           $A_{j+1} \leftarrow key$ 
8  return  $A$ 
```

Exercises

- Write C code for the following algorithm.

Algorithm: MERGE (A, p, q, r)

```
1  $n_1 \leftarrow q - p + 1, n_2 \leftarrow r - q$   
2  $L \leftarrow \text{MAKE-ARRAY}(n_1 + 1), R \leftarrow \text{MAKE-ARRAY}(n_2 + 1)$   
3  $L[1.. n_1] \leftarrow A[p..q], R[1.. n_2] \leftarrow A[q+1..r]$   
4  $L_{n_1+1} \leftarrow \infty, R_{n_2+1} \leftarrow \infty$   
5  $i \leftarrow 1, j \leftarrow 1$   
6 for  $k \leftarrow p$  to  $r$   
7   do if  $L_i \leq R_j$   
8     then  $A_k \leftarrow L_i$   
9          $i \leftarrow i + 1$   
10    else  $A_k \leftarrow R_j$   
11         $j \leftarrow j + 1$ 
```

Exercises

- Give the C code for linear search.
- Give the C code for binary search.

Exercises

- Give the definition of a 7-by-8 array of floating points. How to reference the element at the 3rd row and the 5th column?
- Write a function `addMatrix` to return the sum of a `ROW-by-COL` two-dimensional array of integers, where `ROW` and `COL` are symbolic constants.